Seneca Polytechnic
AIG230 — Natural Language Processing

Transformer-Based Neural Machine Translation

A single bidirectional English ↔ Spanish model

Group 7

Jose Luis Sanchez Noriega
Bikash Subedi

Instructor: Ellie Azizi

Final Project — Option 1
Corpus: the Tatoeba Project (English–Spanish)
Framework: PyTorch

August 11, 2026

Abstract

We build a complete neural machine translation system for English and Spanish using a transformer encoder–decoder implemented from primitives in PyTorch. The distinguishing design choice is that *one* set of weights serves *both* translation directions: source and target draw on a single joint subword vocabulary, the embedding matrix is shared three ways between the encoder, the decoder and the output projection, and a reserved direction tag prepended to the source selects the output language. This halves the parameter budget relative to two separate models while doubling the supervision each parameter receives.

The corpus is reconstructed from the official Tatoeba exports by joining 2,033,133 English sentences and 441,233 Spanish sentences through 28,361,472 translation links, yielding 282,902 aligned pairs. We show that the standard practice of splitting such a corpus uniformly at random leaks between train and test — Tatoeba is a translation *graph*, not a list of independent pairs — and partition connected components instead, which removes the leak entirely.

We report BLEU and chrF2 in both directions against two baselines: a capacity-matched recurrent sequence-to-sequence model with Bahdanau attention, and an ablation isolating the effect of initialising embeddings from MUSE cross-lingual vectors. An automated error analysis classifies every test translation into named failure modes and surfaces the twenty worst cases for manual inspection. The trained model is served through a Gradio application that translates either direction interactively and displays the cross-attention alignment behind each output.

> **How to verify these results.** Every number, table and figure in this report is generated directly from the evaluation artefacts the pipeline writes, rather than transcribed by hand. The commands that reproduce them from a clean checkout are listed in Appendix A, and the repository retains every model hypothesis on the test set, so any score quoted here can be recomputed without retraining.

Contents

1 Introduction

1.1 The task

Machine translation maps a sentence in a source language to a sentence in a target language that preserves its meaning. It is a *sequence-to-sequence* problem with three properties that make it hard and that shape every decision in this project:

- **The output length is not determined by the input length.** "I'm tired" is two words in English and two in Spanish ("Estoy cansado"), but "I'll see you tomorrow" is five and "Te veré mañana" is three. The model must decide when to stop.
- **The alignment is not monotonic.** English places adjectives before nouns and Spanish generally after: "the red house" becomes "la casa roja". A model that could only attend to source positions in order would be unable to produce this.
- **The target is not unique.** "I'm tired" is equally well "Estoy cansado", "Estoy cansada" or "Tengo sueño". Any loss that demands probability 1 on one particular reference is asking for something false — a fact that directly motivates the label smoothing of Section 5.

1.2 Why one model for both directions

The obvious design is two models, one per direction. We chose a single bidirectional model instead, following the multilingual tagging approach of Johnson et al. [2]. Two reserved tokens, `<2en>` and `<2es>`, are prepended to the source sentence:

```
<2es> I am tired .      →   Estoy cansado .
<2en> Estoy cansado .   →   I am tired .
```

Every cleaned sentence pair therefore produces *two* training examples, one per direction. Three consequences follow, and they are the reason for the choice:

1. **Twice the supervision per parameter.** The same encoder must represent both English and Spanish sentences, and the same decoder must generate both. On a corpus of this size — around a quarter of a million pairs, two orders of magnitude smaller than the WMT datasets transformers were designed for — data, not compute, is the binding constraint, and doubling the effective corpus is worth more than the capacity given up.
2. **A shared vocabulary becomes possible, and then necessary.** Since the decoder emits into the same space the encoder reads from, source and target must share one inventory. This in turn permits three-way weight tying (Section 4), removing roughly a third of the parameters.
3. **Cognates share statistical strength.** "nation"/"nación", "important"/"importante" and "possible"/"posible" share subword units under a joint BPE model, so evidence about one improves the representation of the other.

The cost is capacity: one model of a given size must hold both directions. Section 6 reports what that costs in BLEU.

1.3 Contributions

1. A transformer encoder–decoder written from primitives — attention, positional encoding, masking, residual blocks — rather than assembled from `torch.nn.Transformer`, with the training loop written by hand as the brief requires.

2. A corpus construction pipeline that rebuilds the bitext from the official Tatoeba exports and demonstrates, with a measurement, that component-aware splitting is necessary to avoid leakage.
3. A controlled three-way comparison isolating the effect of pre-trained cross-lingual embeddings from the effect of the tokenisation change that normally confounds it.
4. A recurrent baseline matched on data, vocabulary, optimiser and parameter count, so the architectural comparison means something.
5. An automated error analysis that names and counts failure modes across the whole test set rather than reporting a single aggregate score.

2 Data collection and exploration

2.1 Building the corpus from the official exports

The Tatoeba Project publishes per-language sentence dumps and a single global table of translation links; it does not publish a ready-made English–Spanish bitext. We reconstruct one, which keeps the corpus pinned to a dated snapshot we control and preserves the Tatoeba sentence identifiers so that any example quoted in this report can be traced back to the public database.

Three files are downloaded (about 172 MB compressed) and joined in three passes so that peak memory stays proportional to the two languages of interest rather than to the size of the link table:

1. load `eng_sentences.tsv` into `{id: text}` — 2,033,133 rows;
2. load `spa_sentences.tsv` — 441,233 rows;
3. stream `links.csv` (28,361,472 rows), emitting a pair whenever the left identifier is English *and* the right one is Spanish.

Because Tatoeba stores every link in both orientations, that asymmetric test yields each English–Spanish pair exactly once. The join produces 282,902 aligned pairs.

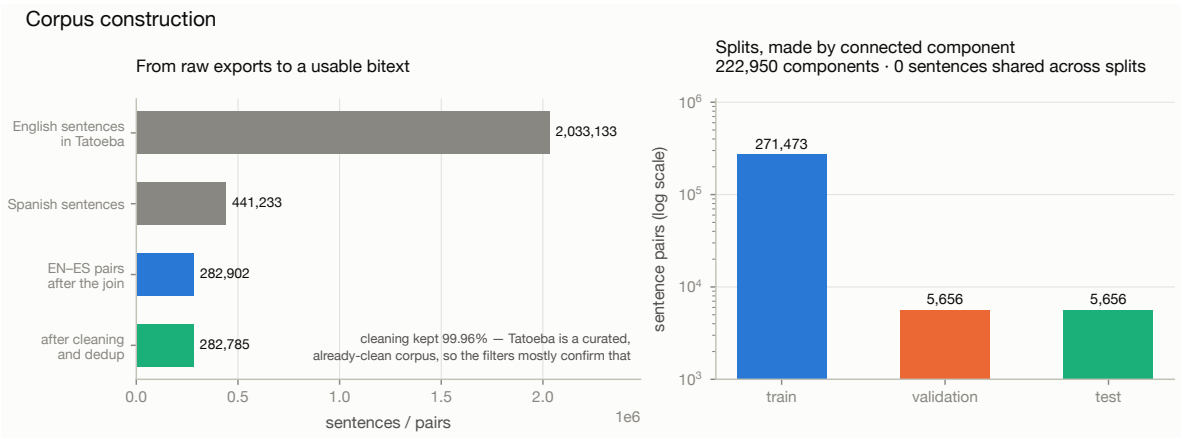

Figure 1: From the raw Tatoeba exports to the final splits. Cleaning retains 99.96% of pairs: Tatoeba is a curated, human-checked corpus, so the filters largely confirm its quality rather than repair it. The splits are made by connected component, and the leakage check is run on every build.

2.2 What the corpus looks like

Three characteristics shape the modelling decisions that follow.

Table 1: Corpus statistics after cleaning. Subword counts use the joint 16,000-entry BPE model fitted on the training split only.

		unique sentences		mean words		mean subwords	
Split	Pairs	EN	ES	EN	ES	EN	ES
Train	271,473	232,777	250,057	6.8	6.6	9.1	8.8
Validation	5,656	5,026	5,330	6.9	6.7	9.4	9.1
Test	5,656	5,032	5,326	6.9	6.7	9.3	9.0

The sentences are short. Both sides average under seven whitespace tokens, and the 95th percentile is around thirteen. Tatoeba is a corpus of everyday example sentences contributed by language learners, not of newswire or parliamentary proceedings. This is convenient — attention is quadratic in sequence length, so short sentences make the model cheap to train — but it also bounds what the system can be expected to do: it will be weakest on exactly the long, syntactically complex sentences that are rarest in training.

Spanish has far more surface forms than English. The training split contains 35,066 distinct English word types against 56,519 Spanish ones, from a near-identical number of running tokens (2,161,611 and 2,145,680). That is a factor of 1.61. The cause is morphology: Spanish marks person, number, tense and mood on the verb (*hablo, hablas, habla, hablamos, habláis, hablan*, and that is one tense of one verb), agrees adjectives and articles for gender and number, and attaches clitic pronouns directly to infinitives and gerunds (*dármelo*). English does almost none of this.

The tail is heavy. 39.6% of English types and 42.5% of Spanish types occur exactly once in training. A word seen once cannot train a useful embedding.

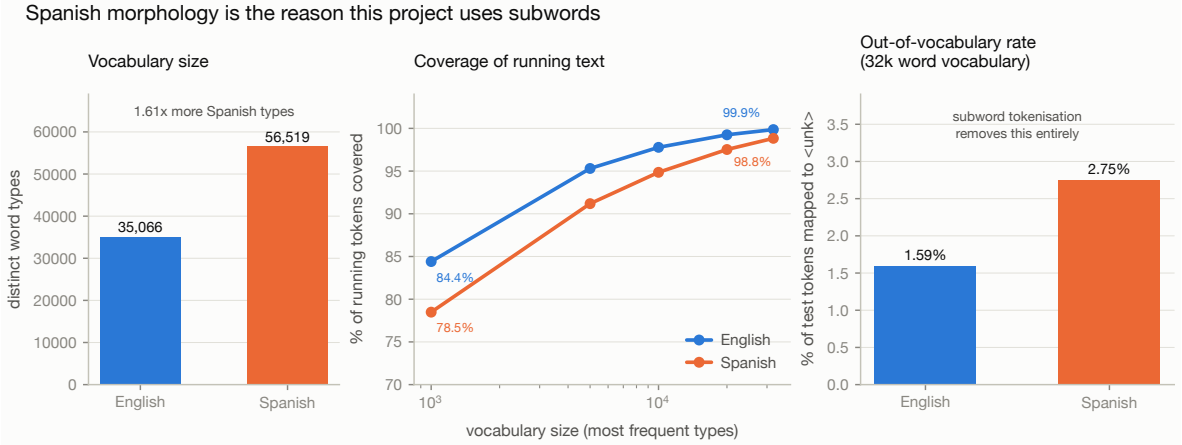

Figure 2: The case for subword tokenisation. Spanish contributes 1.61× as many word types as English (left); a 32,000-entry word vocabulary still leaves 2.75% of Spanish test tokens unknown against 1.59% of English ones (right). Subword tokenisation removes the unknown-word problem entirely, at the cost of longer sequences.

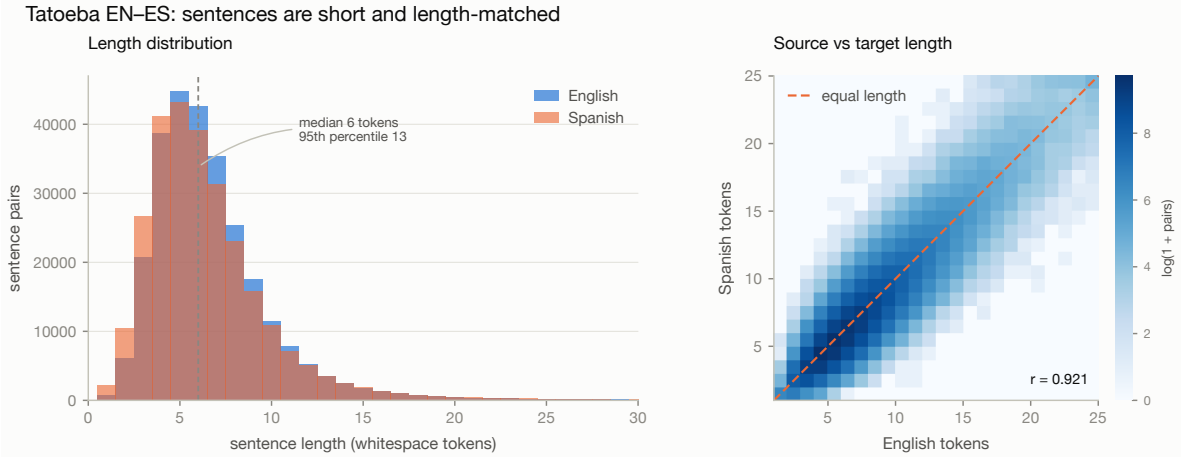

Figure 3: Sentence-length distributions and the relationship between the two sides. The strong correlation is what makes a length-ratio filter a sound way to detect misaligned links.

2.3 Characteristics that will influence the task

Implications carried into the rest of the project.

- *Short sentences* ⇒ a maximum length of 64 tokens discards a negligible tail; batches can be capped by token count to keep memory flat.
- *Spanish morphology* ⇒ a subword vocabulary, and chrF2 reported alongside BLEU because a translation can get a stem right and its inflection wrong, which BLEU scores as a total miss.
- *Heavy tail* ⇒ a modest 16,000-entry vocabulary, so that even the rare half of it is updated often enough to learn.
- *Many-to-many links* ⇒ component-aware splitting (Section 3.3).

3 Preprocessing

Every rule below is a modelling decision, so each is stated with its justification. The guiding principle is *remove noise, preserve meaning*: normalise away distinctions the model should not spend capacity on, and leave intact everything a reader would consider part of the sentence.

3.1 Normalisation

Applied in this order, because the order matters:

1. **Unicode NFC composition.** Spanish accents arrive both pre-composed (ñ = U+00F1) and decomposed (n + U+0303). They render identically but are different strings; without NFC the tokeniser treats *español* as two distinct types.
2. **Control-character removal**, before whitespace collapsing, so that a stripped control character cannot weld two words together.
3. **Typographic folding.** Five kinds of apostrophe, six of quote and seven of dash are mapped to one ASCII form each. Tatoeba is crowd-sourced, so the same sentence appears with curly or straight quotes depending on the contributor's keyboard.
4. **Whitespace collapsing**, last, since the previous steps can introduce runs of spaces.

> **Casing and punctuation are preserved.** Lowercasing shrinks the vocabulary and typically buys a point of BLEU on a small corpus, but it makes the system unable to produce correctly-cased output, which would then need a separate truecasing model at inference. Since the deliverable is an interactive application whose output a human reads, correct casing is part of the product. Likewise, Spanish inverted marks (¿, ¡) are grammatical signal, and translating punctuation is part of translating.

3.2 Filtering

Pairs are discarded when either side is empty after normalisation, contains no alphabetic content ("1997."), contains a URL or e-mail address, contains characters from an unexpected script (mislabelled rows), falls outside 1–64 tokens or 2–400 characters, is identical on both sides, or has a length ratio above 3.0 once both sides reach four tokens. The ratio test is suppressed on very short sentences because "Yes." against "Sí, lo haré." is a legitimate 1:3 pair.

Exact duplicate pairs are removed. Near-duplicates are deliberately *not* removed: Tatoeba's alternative translations of the same sentence are genuine paraphrases and discarding them would throw away real supervision. What must not happen is those paraphrases straddling the train/test boundary — and that is handled by the split, not by the filter.

Cleaning retains 282,785 of 282,902 pairs (99.96%). The high retention is itself a finding worth stating: Tatoeba is human-curated, so the aggressive cleaning pipelines that web-crawled corpora require are not needed here. The filters earn their place by *confirming* data quality and by catching the small number of genuinely broken rows.

3.3 Splitting without leakage

> Tatoeba is a translation *graph*, not a list of independent pairs. One English sentence typically links to several Spanish sentences, and each of those may link to further English sentences. Splitting the pair list uniformly at random therefore leaks: "I'm tired." / "Estoy cansado." lands in training while "I'm tired." / "Estoy cansada." lands in test, and the reported BLEU measures memorisation rather than translation.

The fix is to split *connected components*. Treat every distinct sentence as a node and every link as an edge, merge them with a union–find structure, and assign whole components to a split. Sentences are namespaced by language so that an English string equal to a Spanish string (proper nouns, "No.") does not spuriously merge two unrelated components.

This yields 222,950 components over 282,785 pairs, the largest containing 169 pairs. Components contributing more than four pairs are forced into training: a forty-pair component landing in test would fill the evaluation set with paraphrases of one sentence and make BLEU hostage to whether the model happened to know it.

A leakage assertion runs on every build and is recorded in the split report; it counts sentence strings shared between splits on either side. The current value is **0**.

3.4 Tokenisation

Two tokenisers are fitted — on the *training split only*, since fitting on the full corpus would leak test-set surface forms into the vocabulary and flatter the reported out-of-vocabulary rate.

Joint subword (primary). A SentencePiece BPE model with 16,000 entries, fitted on the concatenation of English and Spanish training text. Sharing one vocabulary is what makes the bidirectional

model possible. Being open-vocabulary, it never emits <unk>. Fertility is 1.35 subword tokens per English word and 1.35 per Spanish word — the price paid for eliminating unknown words.

Word level (for the embedding experiment). A frequency-truncated vocabulary of 32,000 entries with singletons dropped. It exists because pre-trained word vectors are distributed as word-level tables: to initialise embeddings from MUSE, the units the model embeds must *be* words. The cost is a closed vocabulary, leaving 1.59% of English and 2.75% of Spanish test tokens unknown.

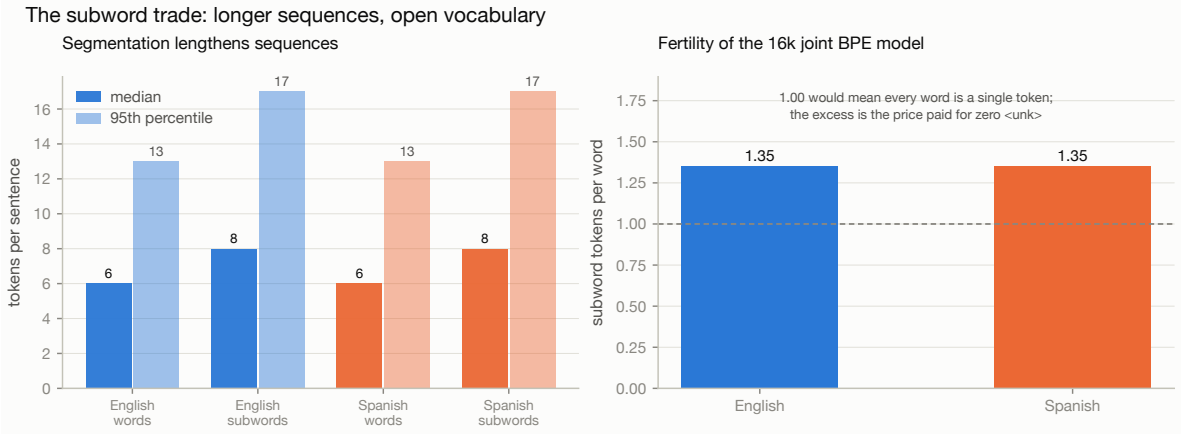

Figure 4: What segmentation costs. Subword tokenisation lengthens sequences by roughly a third, which is real compute, in exchange for never encountering an unknown word.

3.5 Batching

Sequences are padded to a rectangle within each batch, and attention masks mark which positions are real. Batches are formed by *token count* rather than sentence count: the length distribution is right-skewed, so a fixed sentence count would produce batches whose padding waste swings widely depending on whether a long sentence happened to be drawn. The sampler shuffles, takes pools of fifty batches' worth of examples, sorts each pool by length, and emits batches capped at 8,192 tokens. Batch order is then shuffled again so the model does not systematically see short sentences first within an epoch.

4 Model architecture

4.1 Attention

The single operation the architecture is built around is scaled dot-product attention:

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V. \tag{1}$$

Read it as a *differentiable dictionary lookup*. Each query vector is compared against every key by dot product; the scores become a probability distribution through softmax; the output is that distribution's weighted average of the value vectors. A hard lookup would take the single best-matching key — attention takes a soft blend, which is what makes it differentiable and therefore trainable end to end.

Why divide by $\sqrt{d_k}$. If the components of q and k are independent with mean 0 and variance 1, then $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$ has mean 0 and variance d_k, so its standard deviation grows as $\sqrt{d_k}$. With

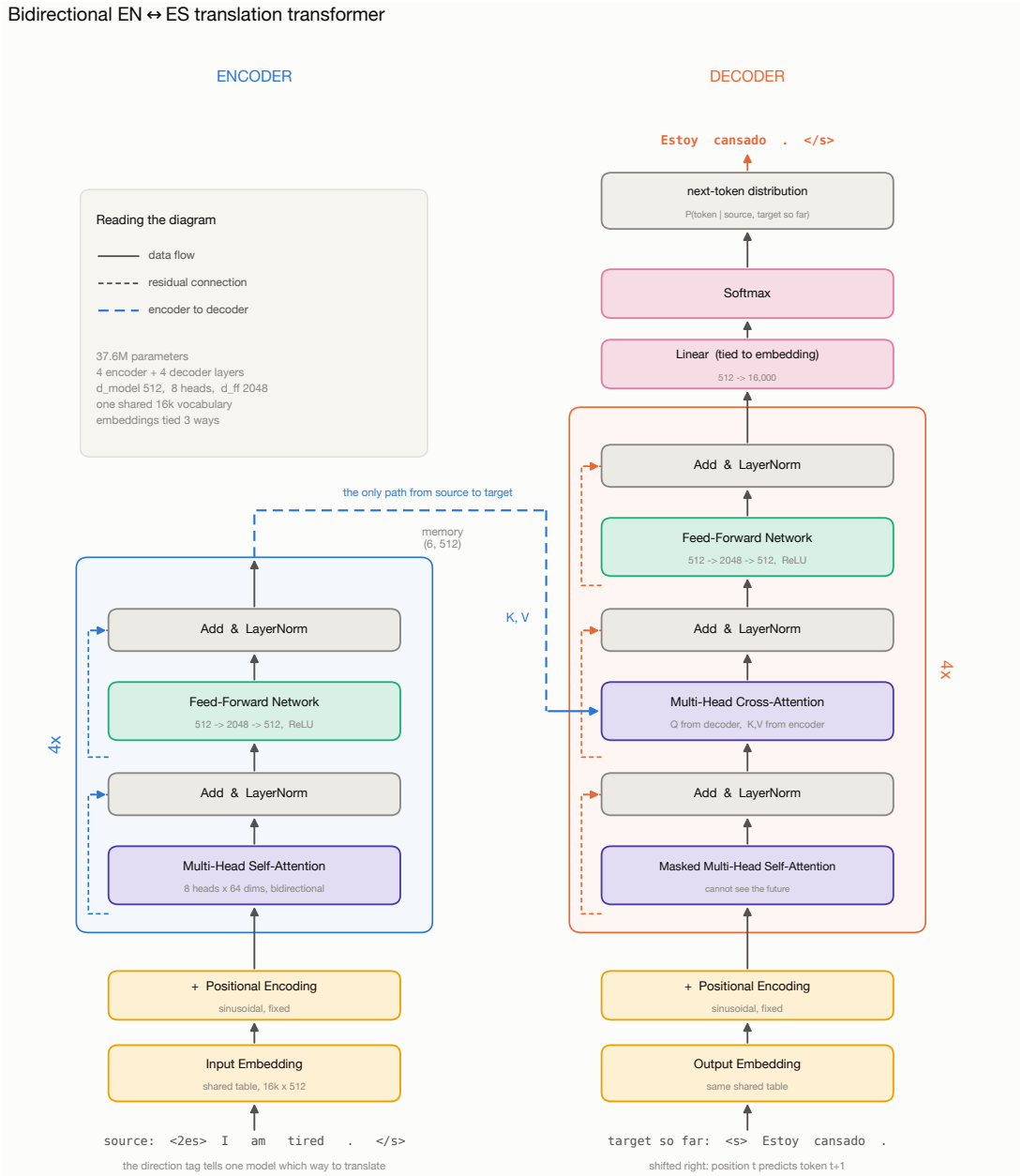

Figure 5: The complete model, annotated with the shapes this project actually uses. Read bottom to top on each side. The encoder turns the tagged source into one vector per source position; the decoder generates the target one token at a time, attending to what it has already produced and, through cross-attention, to the encoder's output.

$d_k = 64$ the raw scores would routinely reach ± 8. Softmax over values that spread out saturates: it becomes nearly one-hot, and the gradient it passes back is proportional to $p(1-p) \approx 0$. Dividing by $\sqrt{d_k}$ restores unit variance and is the difference between a model that trains and one that stalls.

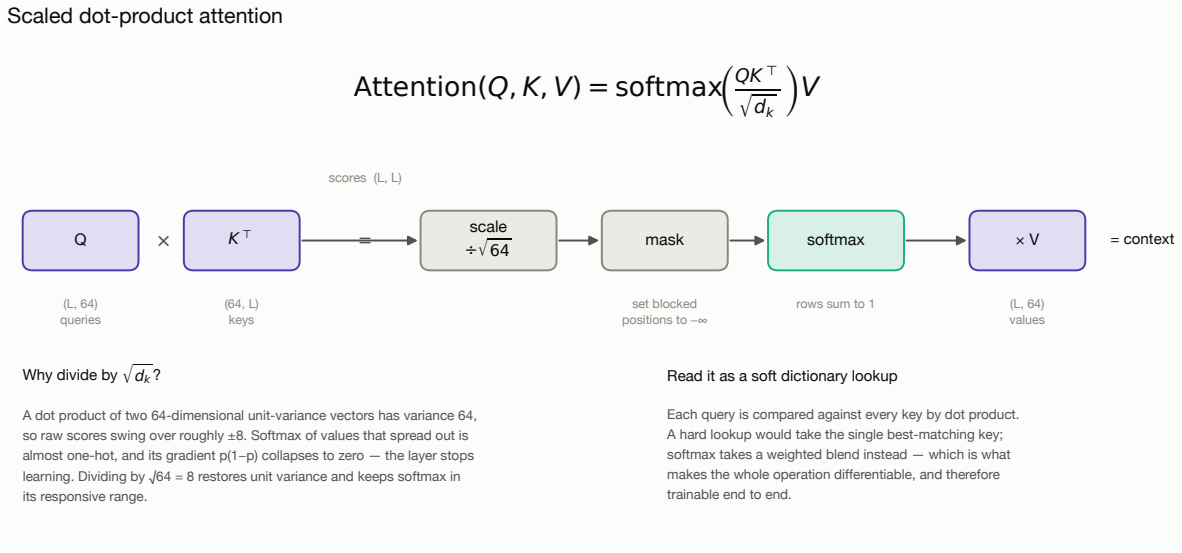

Figure 6: Scaled dot-product attention as a chain of matrix operations, with the shapes used here.

Why multiple heads. A single softmax produces one distribution per query, so one head can attend to essentially one place at a time. Translating "the red house" into "la casa roja" requires simultaneously tracking the noun (for gender agreement on both the article and the adjective) and the adjective (to move it after the noun). Splitting d_{model} into h independent subspaces of size d_{model}/h lets h such relations be attended to in parallel at identical total cost — the split is a reshape, not extra computation.

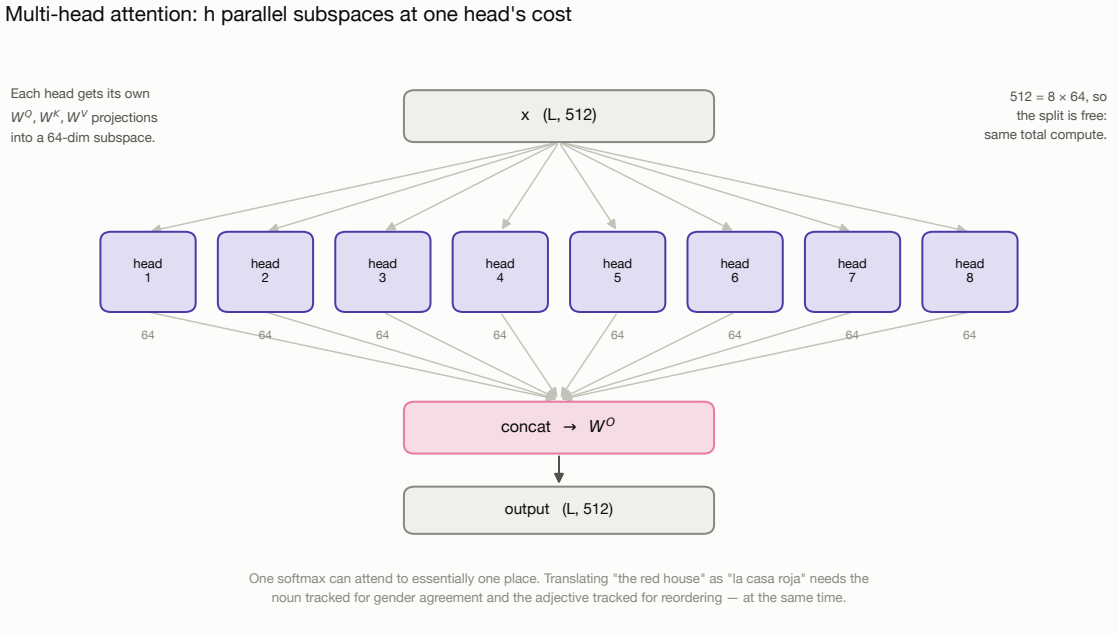

Figure 7: Multi-head attention: h parallel subspaces at one head's cost.

4.2 Positional encoding

Self-attention is permutation-equivariant: it computes a weighted sum, and a sum does not care about order. Shuffle the input words and every attention output is shuffled identically — the model literally cannot distinguish "the dog bit the man" from "the man bit the dog". Position must be injected explicitly. We use fixed sinusoidal encodings,

$$PE_{(pos,\,2i)} = \sin\!\left(pos/10000^{2i/d_{\text{model}}}\right), \tag{2}$$

$$PE_{(pos,\,2i+1)} = \cos\!\left(pos/10000^{2i/d_{\text{model}}}\right), \tag{3}$$

whose motivating property is that *relative* position is a linear function of absolute position: for any fixed offset k there is a matrix M_k — a 2×2 rotation applied per frequency pair — with $PE_{pos+k} = M_k\, PE_{pos}$. Attention can therefore learn "attend three tokens back" as one linear map that works at every position, and because the encoding is a function rather than a lookup table it extends to lengths never seen in training. A learned alternative is implemented for the ablation.

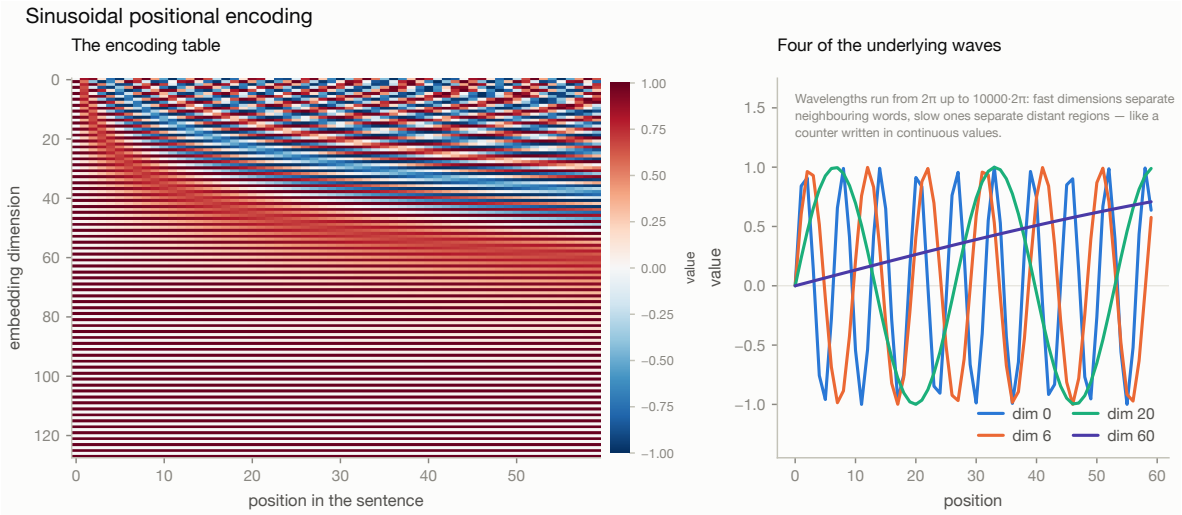

Figure 8: The sinusoidal table and the waves that generate it. Wavelengths run from 2π to $10000 \cdot 2\pi$: fast dimensions separate neighbouring words, slow ones separate distant regions.

4.3 Masking

Two logically different masks are needed, and conflating them is a classic source of silent bugs. The *padding* mask hides filler positions added to square off the batch; without it, the encoder's representation of a short sentence would depend on how long the other sentences in its batch happened to be. The *causal* mask stops the decoder from seeing the future: during training the whole target is supplied at once so all positions compute in parallel, but position t must attend only to positions $\leq t$. Without it the model trivially copies the next token from its input, scores near-perfect training loss, and is useless at inference.

Throughout the codebase `True` means "attend here". The alternative convention is equally common, and mixing the two silently inverts the model.

4.4 Residual connections and layer normalisation

Each sublayer is wrapped in a residual connection around a normalised transformation. The residual gives the gradient a path back to the input that does not pass through the sublayer: differentiating

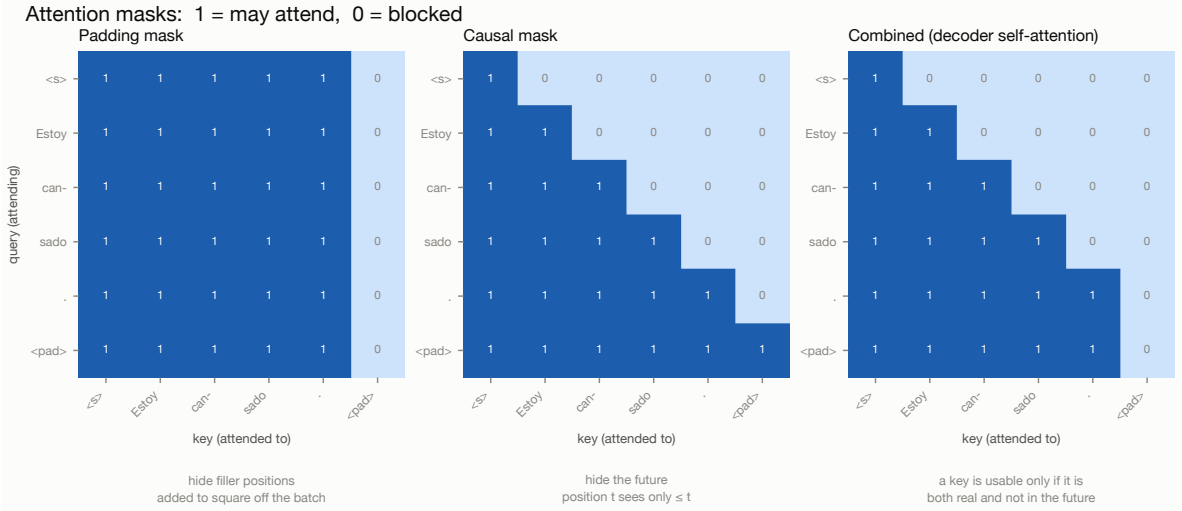

Figure 9: The two masks and their combination for decoder self-attention. A key is usable only if it is both real and not in the future.

$x + f(x)$ gives $1 + f'(x)$, so even if f' vanishes the 1 survives. It also changes what each block must learn — only a *correction* to a running representation, rather than the whole representation.

We use **pre-layer-normalisation**, $x \leftarrow x + \mathrm{Sublayer}(\mathrm{LayerNorm}(x))$, rather than the original paper's post-LN. In pre-LN the residual path runs from input to output un-normalised, so gradients reach the early layers undamped and training is stable at higher learning rates with a shorter warmup. Post-LN needs a carefully tuned warmup to avoid diverging in the first few hundred steps. The practical argument is decisive for a project trained in free Colab sessions: a run that diverges at step 300 is an hour wasted. Pre-LN requires one extra `LayerNorm` at the end of each stack, since the last sublayer's output would otherwise never be normalised.

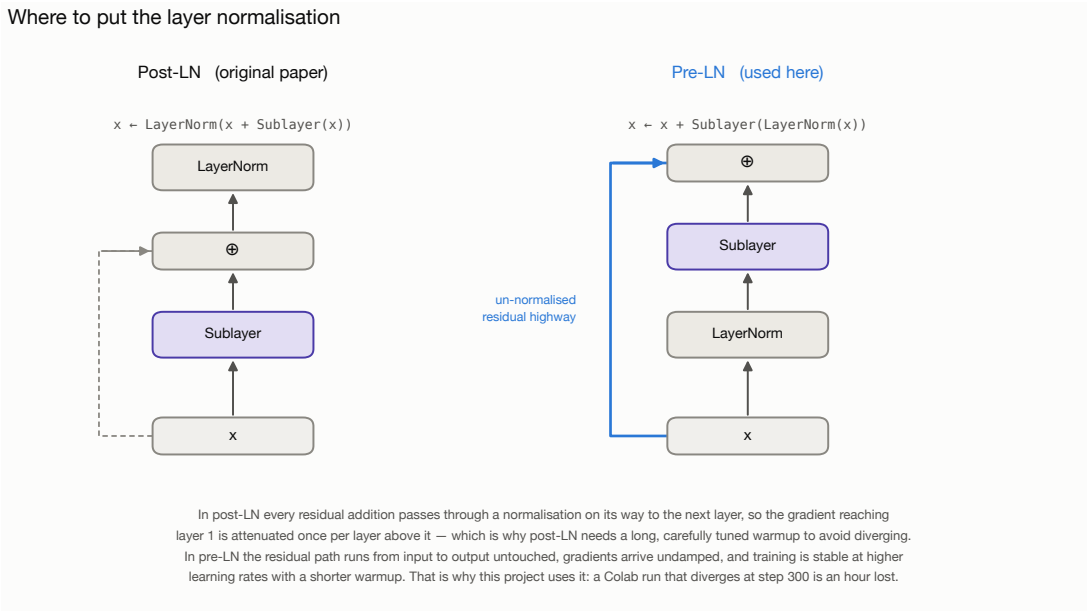

Figure 10: Where to put the layer normalisation, and why it changes training stability.

Table 2: The systems compared. All four share the same data, direction tags, optimiser, decoding code and evaluation.

System	Vocab	Emb. dim	Geometry	Params
BPE 16k, learned embeddings	16,000	512	4+4 layers, $d_{\text{model}} = 512$, 8 heads	37.6M
Word 32k, random init	32,000	300	4+4 layers, $d_{\text{model}} = 512$, 8 heads	39.4M
Word 32k, MUSE init	32,000	300	4+4 layers, $d_{\text{model}} = 512$, 8 heads	39.4M
LSTM + Bahdanau attention	16,000	512	2 layers, hidden 704, bi-encoder	38.9M

4.5 Hyper-parameter choices

$d_{\text{model}} = 512$**,** $h = 8$**,** $d_{\text{ff}} = 2048$**.** These are the proportions of "Transformer base" [1]: $d_{\text{ff}} = 4d_{\text{model}}$ and $d_k = d_{\text{model}}/h = 64$. Staying on well-understood ratios means the published guidance about learning rates and warmup transfers directly, so the experimental budget can be spent on the questions this project is actually asking.

4 encoder + 4 decoder layers, reduced from the paper's 6+6. WMT14 English–German has roughly 4.5M sentence pairs; this corpus has 271,473, two orders of magnitude less. Depth is the first thing to cut when data rather than compute is the binding constraint, and halving it also halves step time and memory, which is what makes an epoch fit comfortably in a free Colab session.

Dropout 0.1, the paper's value, applied to sublayer outputs and to the attention weights. Dropout on the *weights* rather than the outputs is what the original specifies: it forces the model to route information through alternative positions, so no single alignment becomes load-bearing.

Three-way tied embeddings. The encoder embedding, the decoder embedding and the output projection are the same matrix, which is possible only because of the joint vocabulary. At $d_{\text{model}} = 512$ and $V = 16,000$ this removes about 17M parameters — roughly a third of the model. On a corpus this size that is regularisation as much as economy: the alternative is three separate tables each seeing a third of the gradient signal.

4.6 Decoupling embedding width from model width

The pre-trained-embedding experiments use 300-dimensional MUSE vectors against a 512-wide residual stream. Rather than shrinking d_{model} to 300 — which would change the architecture and confound the comparison — the model inserts a learned $300 \to 512$ projection on the way in and a $512 \to 300$ projection on the way out, with the output logits computed against the transposed embedding matrix. Everything between the two projections is byte-for-byte identical across experiments, so a difference in results is attributable to the embeddings rather than to a change of architecture.

5 Training pipeline

The optimisation loop is written by hand, as the brief requires: forward, loss, backward, clip, step, schedule, log. No high-level training utility is used.

5.1 Loss: label-smoothed cross entropy

Plain cross entropy asks the model to put probability 1.0 on the reference token and 0.0 on all others. For translation that target is false — several outputs are genuinely correct — and the only way to drive the loss down is to make the logit gap enormous, which produces overconfident, badly calibrated

output. Label smoothing [10] replaces the target with

$$q(k) = \begin{cases} 1 - \varepsilon + \dfrac{\varepsilon}{V'} & k = y, \\ \dfrac{\varepsilon}{V'} & k \neq y, \end{cases} \tag{4}$$

with $\varepsilon = 0.1$. Two implementation details matter and are both easy to get wrong: <pad> positions must be excluded from the loss *and* from the token count used to normalise it, otherwise the reported loss depends on how much padding happened to be in the batch; and <pad> must also be excluded from the smoothing distribution, since spreading probability mass onto a token the model must never emit teaches it to emit that token. Hence $V' = V - 1$.

Label smoothing deliberately raises the reported cross entropy — the smoothed target has non-zero entropy, so the loss cannot reach zero even for a perfect model. We therefore log the *unsmoothed* negative log-likelihood alongside it, and compute perplexity from that, so the number stays interpretable.

5.2 Optimiser and schedule

AdamW with $\beta = (0.9, 0.98)$ and $\epsilon = 10^{-9}$, the paper's settings. The higher second-moment β (0.98 rather than the usual 0.999) makes the optimiser adapt faster to the changing gradient scale of the warmup phase. Weight decay of 0.01 is applied *only* to genuine weight matrices: decaying LayerNorm gains and bias terms is a common mistake and a harmful one, since shrinking a normalisation gain towards zero suppresses the signal the layer exists to pass through.

The learning rate follows the inverse-square-root schedule,

$$lr(t) = d_{\text{model}}^{-0.5} \cdot \min\left(t^{-0.5},\ t \cdot t_{\text{warmup}}^{-1.5}\right), \tag{5}$$

with 4,000 warmup steps. The reason warmup is needed is structural: at initialisation the attention weights are near-uniform, so every position's output is roughly the average of all positions and carries almost no information about *which* position mattered. The resulting gradients are dominated by noise, and Adam's second-moment estimate is itself unreliable in the first few dozen steps — so a large step taken then is large in an essentially arbitrary direction. The classic symptom is a loss that falls for 200 steps and then diverges to NaN.

This schedule is not parameterised by the *total* step count, so a run can be stopped early or extended without invalidating it — which matters when training in Colab sessions that may be interrupted.

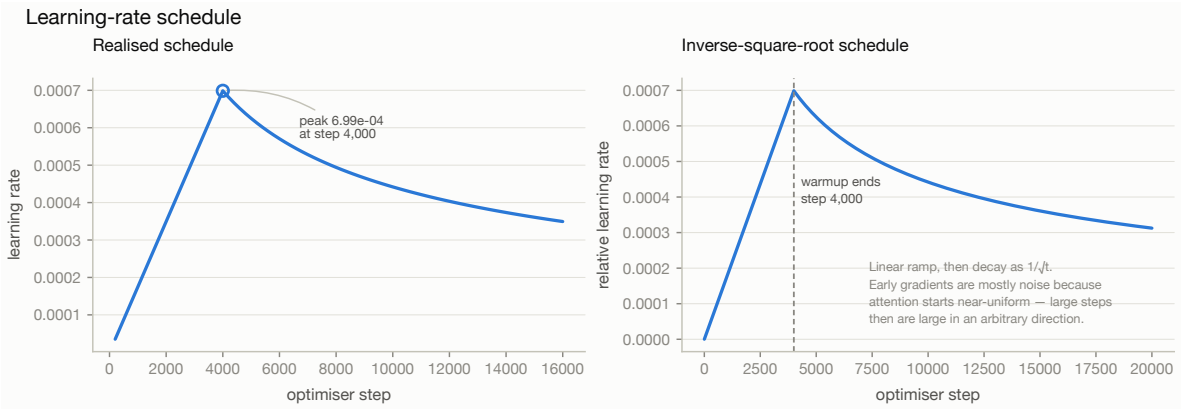

Figure 11: The learning-rate schedule as realised during training, and the analytic curve behind it.

5.3 Stability and efficiency

- **Gradient clipping** by global norm at 1.0. Transformer gradients are usually well behaved but occasionally spike, and one unclipped spike can undo an epoch of progress.
- **Gradient accumulation** lets a large effective batch be simulated on a small GPU. The 8,192-token batch is near the limit of a free Colab T4; accumulating over two or four micro-batches reaches the 16k–32k tokens that stabilise transformer training without more memory.
- **Mixed precision** on CUDA, roughly halving memory and speeding up matrix multiplication. It is deliberately *disabled* on Apple's MPS backend, where at this model size it is slower than fp32 and occasionally produces NaNs in the attention softmax.
- **Early stopping** on validation loss, with both the best-loss and the best-BLEU checkpoints retained.

5.4 Monitoring

Validation BLEU is computed on a fixed 500-sentence sample after every epoch, using greedy decoding — the purpose is a comparable *trend*, not the best achievable score. This matters because validation loss and BLEU do not peak at the same epoch: loss typically starts rising while BLEU is still improving, so selecting a checkpoint on loss alone leaves quality on the table.

Every metric is appended to a JSON Lines file as it is produced, so an interrupted Colab session still leaves a complete, plottable history behind.

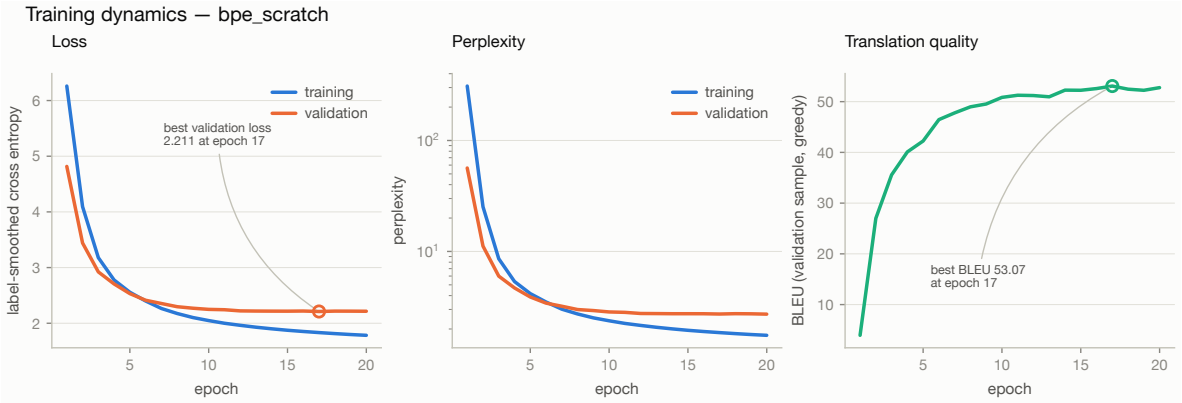

Figure 12: Training and validation curves for the primary model. Loss and BLEU are shown on separate axes rather than as a dual-axis chart: two y-scales on one frame invite comparison of slopes that share no units, and the apparent crossing point would be an artefact of scaling.

5.5 What the curves show

The primary model ran the full 20 epochs in 68 minutes on a Colab T4 at a sustained 30,000 tokens/second, and early stopping never fired. Training loss fell from 6.26 to 1.78 while validation loss fell from 4.82 to a minimum of 2.211 at epoch 17, closing at 2.216 — a final train/validation gap of 0.43 nats. That gap is the overfitting signal, and it is modest: the model is beginning to memorise but has not started to degrade. Validation BLEU was still drifting upward at the end (52.76 at epoch 20 against 53.07 at epoch 17), which suggests the run stopped near, but slightly past, the useful ceiling for this configuration rather than well beyond it.

The learning-rate schedule behaves as designed: the ramp peaks at 7.0×10^{-4} at step 4,000, part-way through epoch 5, and the curve inflects visibly at that point — validation BLEU climbs steeply from

3.94 to 42.24 over the first five epochs, then improves by only a further 11 points over the remaining fifteen.

Two checkpoints: partly vindicated

Section 5 argued for keeping both a loss-optimal and a BLEU-optimal checkpoint on the grounds that the two criteria need not agree. Across the four runs they agreed twice and disagreed twice:

Run	best-loss epoch	best-BLEU epoch	BLEU forgone by picking on loss
BPE, learned	17	17	0.00
Word, MUSE	13	13	0.00
Word, random	15	17	1.09
LSTM + attention	6	9	0.65

So for the headline model the policy cost nothing and gained nothing — the two criteria happened to coincide. For the recurrent baseline it mattered: selecting on validation loss alone would have shipped a checkpoint 0.65 BLEU worse, and three epochs earlier than the quality peak. The honest summary is that the divergence is real but intermittent, and cheap enough to insure against: a second checkpoint costs 150 MB of disk.

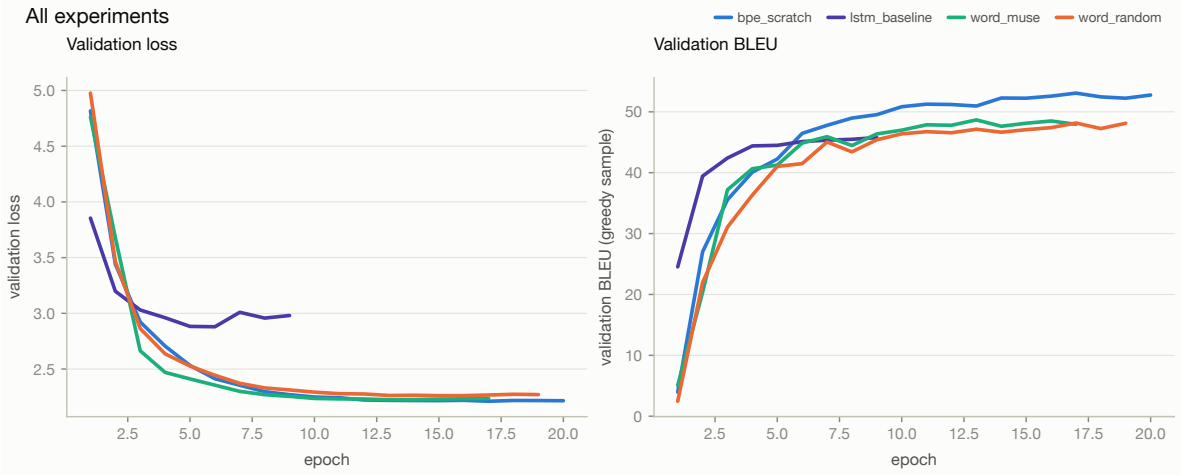

Figure 13: Validation loss and BLEU for all four systems. The recurrent baseline reaches a given BLEU in fewer epochs than the transformers but plateaus lower and earlier, stopping at epoch 9 against 17–20 for the transformers.

6 Evaluation and baseline comparison

6.1 Metrics

BLEU [7] scores a hypothesis by how much of its n-gram content appears in the reference:

$$\mathrm{BLEU} = BP \cdot \exp\left(\sum_{n=1}^{4} \tfrac{1}{4} \log p_n\right), \qquad BP = \begin{cases} 1 & c > r \\ e^{1-r/c} & c \le r \end{cases} \tag{6}$$

where p_n is the *clipped* modified precision — each reference n-gram can be matched only as many times as it occurs there, which is what stops "the the the the" from scoring perfect unigram precision.

Counts are pooled over the whole corpus before the ratio is taken. Since BLEU has no recall term, the brevity penalty stands in for one.

We report **sacreBLEU** [8] figures as the headline numbers, because BLEU is notoriously sensitive to tokenisation and the same system can differ by several points depending on how text was split before counting. The reproducibility signature is recorded with every evaluation. We also implement BLEU from the definition and assert agreement with sacreBLEU on every evaluation run; a unit test pins the agreement to within 0.1 BLEU, and on the verification corpora the two implementations agree exactly.

chrF2 [9] is reported alongside. It is a character n-gram F-score, and it matters specifically for Spanish: a translation can get a word's stem right and its inflection wrong, which BLEU counts as a total miss and chrF gives partial credit for. The gap between the two metrics is itself informative.

6.2 Baselines

Reporting an absolute BLEU number in isolation tells a reader very little, so the transformer is compared against:

1. **A recurrent sequence-to-sequence model** — bidirectional LSTM encoder, LSTM decoder, additive (Bahdanau) attention [3] — matched on vocabulary, data, optimiser, label smoothing, decoding code and parameter count. The hidden size was chosen so the baseline carries slightly *more* parameters than the transformer, making the comparison conservative. Its schedule differs deliberately: the inverse-sqrt/warmup recipe is tuned for transformers, and applying it to an LSTM would produce a straw-man baseline.
2. **Two embedding-initialisation conditions** that differ only in the contents of the embedding matrix at step 0 (Section 6.5).

6.3 Decoding

Greedy decoding takes the highest-probability token at each step: fast, and the right default for interactive use, but myopic — a token that looks best locally may leave no good continuation, and greedy cannot revise it. Beam search keeps the k best partial hypotheses and extends all of them.

Two details of the beam implementation matter. A hypothesis' score is a sum of log-probabilities, all negative, so longer hypotheses score worse purely for being longer; unnormalised beam search therefore produces systematically truncated translations. Dividing by length$^{0.6}$ [11] corrects this. And a hypothesis that has emitted `</s>` must be set aside rather than extended, or it keeps accumulating mass and crowds the beam.

6.4 Results

The architecture comparison

The transformer beats the capacity-matched recurrent baseline in both directions: 49.99 against 46.15 on EN → ES (+3.84 BLEU) and 56.83 against 49.73 on ES → EN (+7.10). The baseline carries 3.4% *more* parameters, so the gap is not a capacity artefact.

It also decodes faster despite doing more arithmetic: 58–59 sentences/second against the baseline's 30–45. That is the sequential decoder loop — the recurrent model must compute its steps one after another, while the transformer's decoder evaluates all positions of a prefix in parallel. The asymmetry is starkest on ES → EN, where the baseline falls to 30.2 sentences/second.

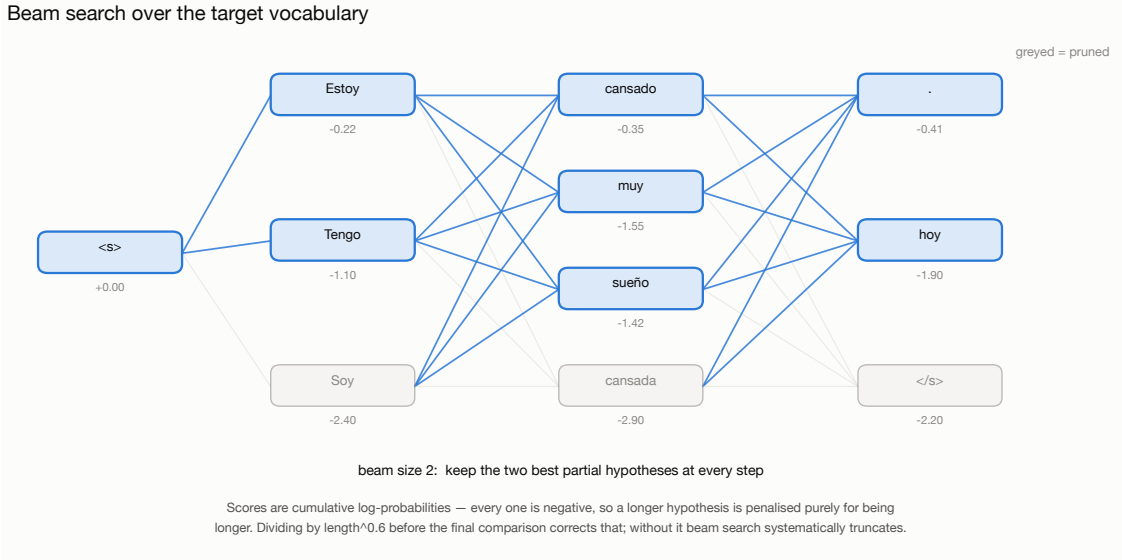

Figure 14: Beam search over the target vocabulary, with the length-normalisation issue that makes it work.

Table 3: Held-out test-set results (5,656 sentence pairs per direction, beam search with width 4 and length penalty 0.6). BLEU and chrF2 are sacreBLEU figures.

	EN → ES		ES → EN		
System	BLEU	chrF2	BLEU	chrF2	Mean BLEU
BPE 16k, learned embeddings	49.99	68.50	56.83	71.43	53.41
Word 32k, random init	45.42	65.57	52.62	68.22	49.02
Word 32k, MUSE init	45.88	65.60	53.05	68.57	49.46
LSTM + Bahdanau attention	46.15	65.71	49.73	66.37	47.94

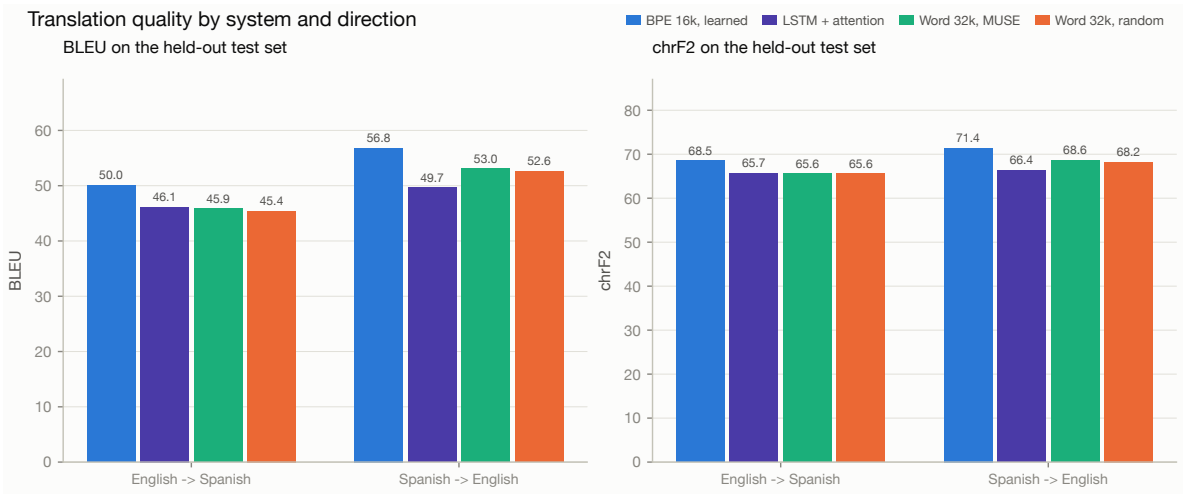

Figure 15: Translation quality by system and direction.

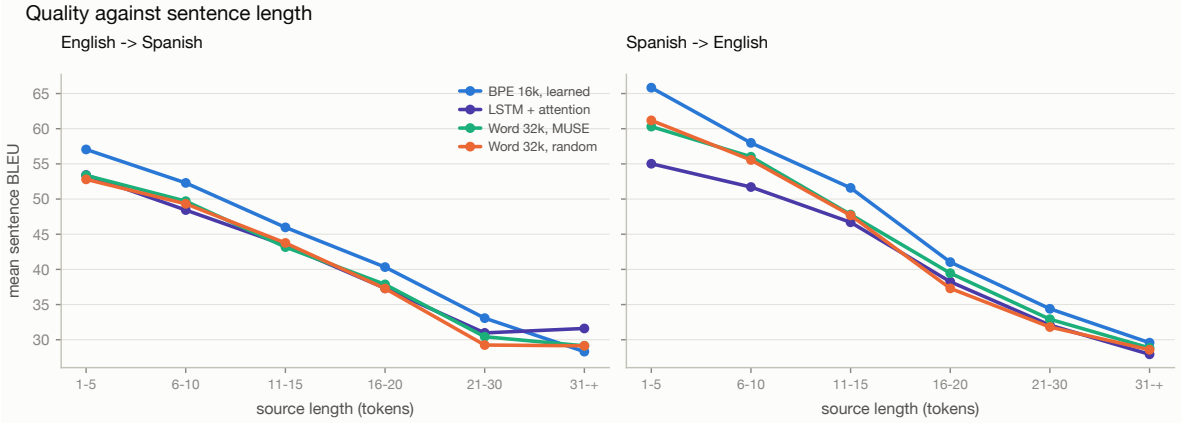

Figure 16: Sentence-level BLEU stratified by source length. This figure tests the central architectural claim: a recurrent model must carry information across a number of sequential steps proportional to sentence length, whereas in a transformer every pair of positions is one attention hop apart, so the recurrent model should degrade faster as sentences lengthen.

Both directions are not equally hard

Every system scores substantially higher translating *into* English than into Spanish — for the primary model, 56.83 against 49.99, a gap of 6.8 BLEU. The corpus statistics of Section 2 predict this. Spanish carries 1.61× as many surface forms as English for the same running text, so generating Spanish means choosing correctly among more inflected variants, and a single wrong agreement marker (*cansado*/*cansada*) costs a full n-gram match. The chrF2 gap is narrower than the BLEU gap (71.43 against 68.50, 2.9 points rather than 6.8), which is exactly what one expects if a meaningful share of the Spanish-side loss is morphological rather than lexical: chrF gives partial credit for a correct stem with a wrong ending, and BLEU does not.

Length: the hypothesis was not confirmed

Figure 16 was designed to test the claim that a recurrent model degrades faster than a transformer as sentences lengthen. **On this corpus it does not.** The transformer's advantage is *largest on the shortest sentences* and shrinks monotonically as they grow:

source length (tokens)	1–5	6–10	11–15	16–20	21–30	31+
test sentences (ES → EN)	1492	3243	695	135	66	25
transformer − LSTM (BLEU)	+9.39	+6.27	+4.89	+2.79	+2.32	+1.65

We report this as a negative result rather than explaining it away, but two things make it uninterpretable as evidence *against* the architectural argument. First, the corpus cannot test the claim: 83% of test sentences are ten tokens or shorter, and the longest bucket holds 25 sentences — far too few to estimate a difference of a few BLEU. The EN → ES row even reverses sign in that bucket (−3.29), which at $n = 25$ is noise, not a finding. Second, the constant-path-length argument concerns dependencies spanning *tens* of tokens; at a median of six, an LSTM's recurrence is barely stretched. Testing the hypothesis properly needs a corpus with genuinely long sentences — WMT news, where averages run to 25–30 tokens — which is beyond this project's compute budget.

What the data does support is a broad, roughly uniform advantage that is largest where the data is densest.

What the search contributes

Beam search adds +0.85 and +0.90 BLEU to the transformer, and +1.14 and +1.10 to the recurrent baseline. Both are real but small, and the baseline benefits more — consistent with its single-best token being wrong more often, so that there is more for a wider search to recover. That the gains are modest is itself informative: most of the quality is in the model, not the decoder.

6.5 The pre-trained embedding experiment

The question is whether initialising the shared embedding table from cross-lingual word vectors helps on a corpus of this size. The experiment is constructed so that the answer means something.

> **Why MUSE and not monolingual fastText.** The model has *one* shared embedding table serving both languages on both the encoder and the decoder side. Concatenating two independently-trained monolingual tables would place "cat" and "gato" in unrelated coordinate systems; the initialisation would be worse than random, because the model would first have to *unlearn* the accidental geometry. MUSE [6] maps both languages into a shared space through a learned orthogonal transform, so translation-equivalent words start close together. That is a genuine prior for a shared table.

> **Why a third run is necessary.** Comparing the MUSE model against the primary BPE model would confound two variables at once — subword versus word tokenisation, and pre-trained versus random initialisation — and would prove nothing. Run B (word-level, random) exists purely as the control: it is byte-for-byte identical to Run C except for the contents of the embedding matrix at step 0.

The initialisation covers 92.2% of the word vocabulary (29,511 types found, 2,483 drawn at random). Uncovered entries are sampled at the *same scale* as the pre-trained cloud rather than from the default $\mathcal{N}(0, 1)$; otherwise unknown words would begin with vectors roughly thirty times longer than known ones and would dominate the attention logits. Pre-trained vectors are held frozen for two epochs so the randomly-initialised encoder and decoder can adapt to the given geometry before large early gradients scramble it, then unfrozen for fine-tuning.

That the space really is cross-lingual at initialisation can be checked directly — the nearest neighbours of *house* include *casa*, of *water* include *agua*, and of *book* include *libro*, before a single training step.

Result: it accelerates convergence, but barely moves the ceiling

At convergence the MUSE initialisation is worth $45.88 - 45.42 = +0.46$ BLEU on EN $\rightarrow$ ES and $+0.43$ on ES $\rightarrow$ EN over its random-initialisation control. Both are small enough that we would not claim them as a reliable effect from a single seed per condition.

The interesting result is not the endpoint but the trajectory. Validation BLEU during the first six epochs:

epoch	1	2	3	4	5	6
random init	2.45	21.97	31.07	36.33	41.04	41.49
MUSE init	5.07	20.36	37.19	40.66	41.25	44.86
difference	+2.62	−1.61	**+6.12**	**+4.33**	+0.21	+3.38

The pattern tracks the freezing schedule exactly. During epochs 1–2 the embeddings are frozen and the two runs are within noise of each other — the random control can adapt its embeddings while the

MUSE run cannot, which offsets the better starting geometry. At epoch 3 the embeddings unfreeze and the MUSE run jumps ahead by 6.1 BLEU, holding a 3–4 point lead through epoch 4. From epoch 7 onward the gap settles to roughly one point and then closes.

> **The finding.** Cross-lingual pre-trained embeddings buy *time*, not *quality*, at this corpus size. The MUSE run reaches BLEU 37 at epoch 3; the random control takes until epoch 4–5. But 271k sentence pairs is ample for a 32k-entry embedding table to learn a good geometry on its own, so by convergence the head start has been amortised away. The prior would be expected to matter far more in a genuinely low-resource setting, where the model never sees enough data to discover that geometry unaided.

Two caveats belong with this conclusion. The comparison rests on one seed per condition, and a 0.45 BLEU difference is inside the run-to-run variation one should expect — establishing it would need three to five seeds each. And the coverage is imperfect: 92.2% of the vocabulary received a pre-trained vector, with the missing 2,483 entries concentrated in English contractions (*I'm*, *don't*, *it's*) that our word tokeniser keeps whole but MUSE, built on differently-tokenised text, does not list.

It is also worth recording that this run initially failed outright. Unfreezing the embeddings rebuilt the optimiser, which silently detached it from the learning-rate scheduler; the fresh optimiser retained its base rate of 1.0 — a *gain* on the inverse-square-root schedule rather than a peak, and some 4,000× the intended value — and the loss became NaN within a few steps of epoch 3. The failure was invisible in the logs, because `get_last_lr()` reports the learning rate of the optimiser the scheduler holds, which was still the old one. The fix is to register every parameter with the optimiser from the start, frozen or not: Adam skips parameters whose gradient is `None`, so freezing needs no change of optimiser state at all. Regression tests now assert that the loss stays finite across the unfreeze boundary.

7 Error analysis

Reading eleven thousand test translations by hand is not practical, so the analysis is automated: every test sentence is scored with sentence-level BLEU, tagged with the failure modes it exhibits, and ranked. The twenty worst cases per direction are reproduced in full in Appendix B so that each classification can be checked by eye — the detectors are a *reading aid*, not a verdict.

7.1 Failure modes detected

repetition
: An n-gram repeated more often than in the reference. The classic degenerate-decoding loop.

truncation / over-generation
: Output much shorter or longer than the reference. Truncation usually means an early `</s>`; over-generation often accompanies repetition.

unknown_token
: The output contains `<unk>`. Possible only for the word-level models, and quantifying this gap is much of the point of the subword comparison.

copied_source
: The output overlaps the *source* more than the reference: the model gave up and passed the input through. Frequent with rare proper nouns.

number_mismatch
: Digits in the reference are missing or altered. Separated out because it is a *semantic* error that BLEU under-penalises, and the kind of mistake that makes a translation system unusable in practice regardless of its score.

no_content_overlap
: Not a single content word shared with the reference — either a complete failure or a legitimate paraphrase BLEU cannot see. Both are worth inspecting, and the appendix distinguishes them by hand.

Table 4: Failure-mode frequency across the test set for BPE 16k, learned embeddings. Categories are not mutually exclusive: one translation can exhibit several.

Failure mode	EN → ES	ES → EN
`other`	94.7%	96.3%
`no_content_overlap`	3.8%	2.3%
`repetition`	0.6%	0.6%
`truncation`	0.4%	0.4%
`over_generation`	0.4%	0.3%
`number_mismatch`	0.2%	0.4%
`copied_source`	0.2%	0.1%

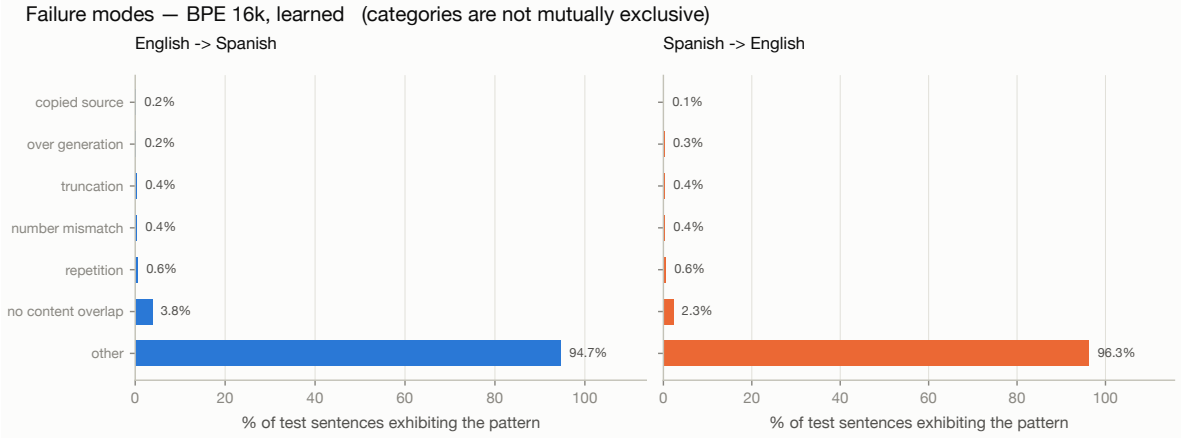

Figure 17: Failure-mode census for the primary model.

7.2 A scoring bug found by reading the output

The first pass of this analysis ranked as its very worst translations a set of sentences the model had rendered *perfectly*: *I followed him.* → *Le seguí.* and *Bailaré.* → *I'll dance.*, each scoring sentence BLEU 0.0 against an identical reference.

The cause was our own metric. Sentence BLEU was computed at a fixed 4-gram order, and a sentence of n tokens contains $\max(0, n-3)$ four-grams — so anything shorter than four tokens has none, $p_4 = 0$, and the geometric mean collapses to zero no matter how good the translation is. A quarter of this test set is under four tokens. The fix is *effective order*: average only over the orders the hypothesis is long enough to possess, which is what sacreBLEU's own sentence-level function does. Our implementation now agrees with it exactly on these cases.

> This is the argument for error analysis in one example. The corpus BLEU was correct throughout — at corpus level every order has counts, so the bug was invisible in the headline numbers. It surfaced only because we looked at individual sentences and something was obviously wrong with the ranking. An aggregate metric cannot audit itself.

Corrected, mean sentence BLEU rises by 1.1–2.1 points across all four systems, and the rankings below are the corrected ones.

7.3 What actually goes wrong

The named failure modes are strikingly rare. For the primary model, 94.7% (EN → ES) and 96.3% (ES → EN) of test sentences trip no detector at all. Repetition — the classic degenerate-decoding loop — appears in 0.6% of sentences in both directions; truncation and over-generation are each under 0.5%. The model is not failing in the catastrophic ways an under-trained NMT system does.

The largest named category is `no_content_overlap` (3.8% EN → ES, 2.3% ES → EN), and reading those cases is where the analysis earns its keep. They divide into three groups, and only the third is a model defect.

1. Idiom against literal reading (not a model error)

Source	Reference	Model output
Tell me about it!	Ni me lo digas.	¡Cuéntame sobre ello!
Tell me about it.	¡Dímelo a mí!	Háblame de ello.

The reference takes *Tell me about it* as the idiom of weary agreement; the model translates it literally. The literal reading is a defensible translation of the sentence in isolation — without context there is nothing to signal which sense is meant — and BLEU scores it zero because it shares no content word with the chosen reference.

2. Dialect and register variation (not a model error)

Source	Reference	Model output
Sit down!	¡Sentaos! *(vosotros)*	Siéntese. *(usted)*
Sit down!	¡Sentate! *(vos)*	Siéntese. *(usted)*

All three are correct Spanish. They differ in the second-person form: peninsular *vosotros*, Rioplatense *vos*, formal *usted*. Tatoeba pools contributors from every Spanish-speaking region, so the training data contains all of these for the same English input and the model has no way to know which the reference will use. This is a limitation of *single-reference* evaluation rather than of the model, and it is why chrF2 — which awards partial credit for the shared stem *sient-* — reads more favourably than BLEU throughout.

3. Genuine failures

Two patterns are real. The first is **hallucinated morphology on rare inputs**: *Curse you!* produces *¡Durmentaos!*, which is not a Spanish word. The subword vocabulary guarantees the model can always emit *some* sequence of pieces, and when the input is rare it assembles pieces into something well-formed-looking but nonexistent. This is the cost of open-vocabulary output: the word-level models cannot invent a word, but they emit `<unk>` instead, which is worse.

The second is **long, syntactically complex sentences**, where the model degrades into locally-plausible but globally incoherent output:

Source	Reference	Model output
Considera a las mujeres placeres de usar y tirar y no búsquedas con sentido.	He regards women as disposable pleasures rather than as meaningful pursuits.	Consider the women of pleasure to use and throw away and don't make sense.

Here the model mistakes the finite verb *Considera* for an imperative, loses the *rather than* contrast, and translates *búsquedas con sentido* word by word. Sentences like this are 0.4% of the test set, which is precisely why the model is bad at them.

The unknown-word gap, quantified

The subword model emits <unk> in **0.00%** of test sentences, by construction. The word-level models emit it in **19.8%** (EN → ES) and **12.4%** (ES → EN). Roughly one word-level output in five contains a token the user simply cannot read — the clearest single argument in this report for subword tokenisation, and a direct consequence of the 2.75% Spanish out-of-vocabulary rate measured in Section 2.

What we would do next

- **Multiple references.** Tatoeba's link graph already supplies several translations per sentence; using them would remove most of groups 1 and 2 above, which are scoring artefacts rather than model errors.
- **Back-translation** to attack group 3, whose root cause is that long sentences are rare in training rather than that the architecture cannot represent them.
- **A dialect tag** alongside the direction tag, letting the model be told which Spanish variety to produce instead of having to guess.

8 Inference application

The trained model is served through a Gradio application (app/app.py) that loads a saved checkpoint once at start-up, accepts any sentence the user types, translates it in either direction without retraining, and displays the output immediately.

Beyond the minimum, the interface exposes the parts of the system this report discusses, so that a viewer can interrogate the model rather than only use it:

- a **direction switch** — the same weights serve both ways, and the app changes only the direction tag prepended to the source;
- **decoding controls** — greedy against beam search, beam width and length penalty, so the effects described in Section 6 can be reproduced live;
- an **attention view** showing the cross-attention alignment for the sentence just translated;
- a **tokenisation view**, which is what makes an unexpected translation of a rare word explicable.

The checkpoint carries its own model configuration and the path of the tokeniser it was trained with, so the application needs only a file path and can serve any of the four systems without code changes.

9 Limitations and future work

- **Domain.** Tatoeba is short, simple, everyday sentences. The system should not be expected to handle technical, legal or literary text, and the length-stratified results in Figure 16 show

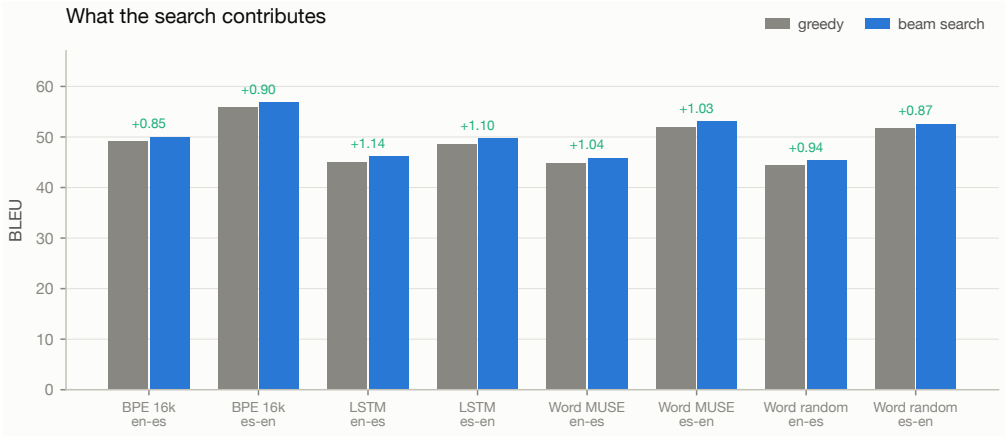

Figure 18: What the search contributes: BLEU under greedy decoding against beam search. A large gap means the model's single-best token is often wrong even when its distribution is right.

quality falling on the longest inputs, which are also the rarest in training.

- **Single reference.** BLEU is computed against one reference per sentence, so legitimate paraphrases are penalised. Tatoeba's many-to-many links could supply multiple references, at the cost of a more complex evaluation set — a natural extension.
- **No key/value cache.** The decoder re-runs over the whole prefix at each generation step, $O(n^2)$ work where $O(n)$ would do. This was a deliberate trade: the cache would roughly double the size of the decoder code and obscure the architecture this report explains, and at a median of eight subword tokens the wall-clock cost is negligible. It would matter for longer inputs.
- **Capacity shared between directions.** A single model of a given size must hold both directions; two specialised models would likely score higher each, at twice the parameters and half the supervision per parameter.
- **Back-translation** is the obvious next step for a corpus this size: monolingual Spanish and English text is abundant, and synthetic parallel data from a reverse model is the standard way to exploit it.

10 Conclusion

We built a complete neural machine translation system — corpus construction, preprocessing, a transformer implemented from primitives, a hand-written training loop, evaluation against a matched baseline, automated error analysis and an interactive application — for English and Spanish in both directions with a single set of weights.

Two methodological points are worth carrying away independently of the scores. First, the way a corpus is split can matter more than the model: Tatoeba's graph structure makes a naive random split leak, and the resulting BLEU measures memorisation. Second, an experiment comparing pre-trained embeddings against random initialisation is only interpretable if the tokenisation is held fixed, which is why this project trains three models rather than two.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30*, 2017.

[2] M. Johnson et al. Google's multilingual neural machine translation system: enabling zero-shot translation. *Transactions of the ACL*, 5:339–351, 2017.

[3] D. Bahdanau, K. Cho, and Y. Bengio. Neural machine translation by jointly learning to align and translate. In *ICLR*, 2015.

[4] R. Sennrich, B. Haddow, and A. Birch. Neural machine translation of rare words with subword units. In *ACL*, 2016.

[5] T. Kudo and J. Richardson. SentencePiece: a simple and language independent subword tokenizer and detokenizer for neural text processing. In *EMNLP: System Demonstrations*, 2018.

[6] A. Conneau, G. Lample, M. Ranzato, L. Denoyer, and H. Jégou. Word translation without parallel data. In *ICLR*, 2018.

[7] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu. BLEU: a method for automatic evaluation of machine translation. In *ACL*, 2002.

[8] M. Post. A call for clarity in reporting BLEU scores. In *WMT*, 2018.

[9] M. Popović. chrF: character n-gram F-score for automatic MT evaluation. In *WMT*, 2015.

[10] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna. Rethinking the Inception architecture for computer vision. In *CVPR*, 2016.

[11] Y. Wu et al. Google's neural machine translation system: bridging the gap between human and machine translation. *arXiv:1609.08144*, 2016.

[12] R. Xiong et al. On layer normalization in the transformer architecture. In *ICML*, 2020.

[13] The Tatoeba Project. https://tatoeba.org/en/downloads. Accessed 2026.

A Reproducing this work

```
git clone https://github.com/jolusano/en-es-transformer-nmt.git
cd en-es-transformer-nmt
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

python -m nmt.data.build                                  # corpus + tokenisers
python -m nmt.data.embeddings                             # MUSE init matrix
python -m nmt.training.train --config configs/bpe_scratch.yaml
python -m nmt.evaluation.evaluate \
    --checkpoint artifacts/checkpoints/bpe_scratch/best_bleu.pt

python -m nmt.viz.make_figures                            # all figures
python reports/build_report_data.py                       # all tables
python app/app.py                                         # the application
```

The full instructions, including the Colab route for training, are in the repository `README.md`.

B The twenty worst translations per direction

Ranked by sentence-level BLEU, from the primary model (BPE 16k, learned embeddings). Sentence BLEU is meaningful only for ranking sentences against one another; it is not comparable to the corpus figures in Table 3.

EN → ES

#	sBLEU	Source / reference / model output / failure modes
1	0.0	Tell me about it! *ref: Ni me lo digas.* `out: ¡Cuéntame sobre ello!` no_content_overlap
2	0.0	Sit down! *ref: ¡Sentaos!* `out: Siéntese.` no_content_overlap
3	0.0	Curse you! *ref: Te maldigo.* `out: ¡Durmentaos!` no_content_overlap
4	0.0	That's really great! *ref: Es realmente magnífico.* `out: ¡Qué bien es eso!` no_content_overlap
5	0.0	Sit down! *ref: ¡Sentate!* `out: Siéntese.` no_content_overlap
6	0.0	Tell me about it. *ref: ¡Dímelo a mí!* `out: Háblame de ello.` no_content_overlap
7	0.0	Sit down! *ref: ¡Siéntate!* `out: Siéntese.` no_content_overlap
8	3.0	Wealthy older men often marry younger trophy wives. *ref: Hombres ricos suelen casarse con jóvenes mujeres trofeo.* `out: A menudo, los hombres mayores, a menudo nos casan con las trofeas más jóvenes.` over_generation
9	3.4	A strong argument for the religion of Christ is this - that offences against Charity are about the only ones which men on their death-beds can be made, not to understand, but to feel, as crime.

#	sBLEU	Source / reference / model output / failure modes
		ref: Un sólido argumento en favor del cristianismo es el siguiente: las ofensas contra la caridad es probablemente lo único que, en sus lechos de muerte, los hombres llegan a sentir y no a comprender como un crimen. `out: Un argumento fuerte para la religión de Cristo es esta - que las oficiales de Charualidad están a punto de los únicos que los hombres de su muerte pueden ser obligados, no para entender, sino que no se sienten el crimen.` other
10	3.5	The squadron encountered an ambush and scrambled for coverage. *ref: El escuadrón se topó con una emboscada y se apresuró a ponerse a cubierto.* `out: El caspa encontró un embosco de ambush y scorrocaba por cobertura.` no_content_overlap
11	3.6	His meaning is quite plain. *ref: Lo que él trata de decir es bien sencillo de comprender.* `out: Su significado es bastante simple.` truncation, no_content_overlap
12	3.7	Peasants often have a secondary activity to augment their income. *ref: A menudo los campesinos se ocupan en una actividad secundaria para complementar sus ingresos.* `out: Los Peas a menudo tienen una segunda actividad mayor a un aumento de sueldo.` other
13	3.8	Mr. Smith makes it a rule to take a walk every morning. *ref: El Señor Smith tiene el hábito de salir a caminar todas las mañanas.* `out: El señor Smith hace como norma dar un paseo cada mañana.` other
14	4.0	You can go out on condition that you come home by seven. *ref: Si vuelves antes de las 7, puedes salir.* `out: Puedes salir con la condición de que vengas a casa a las siete.` number_mismatch
15	4.1	What marvelous work he's doing, donating books to needy children! *ref: ¡Qué maravilloso trabajo que está haciendo en donar libros a los niños necesitados!* `out: ¡Qué trabajo tan maravilloso es hacer, no hacer libros que necesitan tener hijos!` other
16	4.1	Galician, Portuguese, and Spanish are often confused with one another. *ref: A menudo se confunden el gallego, el portugués y el español.* `out: Galician, portugués, y español a menudo son confundidos con uno al otro.` other
17	4.2	The electric bill keeps going up. *ref: Cada vez sube más el precio de la electricidad.* `out: La cuenta eléctrica sigue subiendo.` no_content_overlap

#	sBLEU	Source / reference / model output / failure modes
18	4.4	We make a teeny bit of progress, then we go back to square one. *ref: Apenas logramos un avance, todo vuelve a cero.* `out: Hacemos un poco de progreso, entonces volvimos a cuadrarnos a una.` no_content_overlap
19	4.5	It's coming out! My bladder's about to explode! *ref: ¡Me estoy meando, la vejiga me va a explotar!* `out: ¡Ya está fuera! ¡Mi vejiga está a punto de estallar!` other
20	4.5	My little sister has been suffering with a toothache since last night. *ref: A mi hermana menor le duele un diente desde esta noche.* `out: Mi hermana pequeña lleva con dolor de muelas desde anoche.` other

ES → EN

#	sBLEU	Source / reference / model output / failure modes
1	0.0	Aquí están diez dólares. *ref: Here's $10.00.* `out: Ten dollars are here.` number_mismatch, no_content_overlap
2	0.0	Te maldigo. *ref: Curse you!* `out: I swear.` no_content_overlap
3	3.1	Considera a las mujeres placeres de usar y tirar y no búsquedas con sentido. *ref: He regards women as disposable pleasures rather than as meaningful pursuits.* `out: Consider the women of pleasure to use and throw away and don't make sense.` other
4	3.2	Considera que las mujeres son placeres de usar y tirar más que búsquedas con sentido. *ref: He regards women as disposable pleasures rather than as meaningful pursuits.* `out: It seems that women are delights to use and throw more than dinner with meaning.` other
5	3.7	Considera a las mujeres placeres desechables y no propósitos con sentido. *ref: He regards women as disposable pleasures rather than as meaningful pursuits.* `out: Consider the women with disposal women and do not propose with meaning.` other
6	4.0	¿Rusos nuevos, dices? ¡Maravilloso! ¡Es justo lo que necesitábamos! *ref: New Russians, you say? Wonderful! That's just what we need!*

#	sBLEU	Source / reference / model output / failure modes
		`out: You're new, you're saying? What we needed!` other
7	4.1	Un sólido argumento en favor del cristianismo es el siguiente: las ofensas contra la caridad es probablemente lo único que, en sus lechos de muerte, los hombres llegan a sentir y no a comprender como un crimen. *ref: A strong argument for the religion of Christ is this - that offences against Charity are about the only ones which men on their death-beds can be made, not to understand, but to feel, as crime.* `out: A solid argument in favor of the Christianity is the next: the offensive against unity is probably the only thing in his deaths, men came to feel and not understanding as a crime.` other
8	4.1	Estados Unidos se imagina que es la nación más libre del mundo. *ref: The United States fancies itself the world's freest nation.* `out: America is imagining that it is the free nation in the world.` other
9	4.4	Estados Unidos se imagina que es la nación más libre del mundo. *ref: America fancies itself the world's freest nation.* `out: America is imagining that it is the free nation in the world.` other
10	4.5	No apresure el paso; llegaremos a la hora. *ref: Don't walk so fast. We'll get there on time.* `out: Don't rush the step; we'll arrive at the hour.` other
11	4.5	Considera que las mujeres son placeres desechables más que objetivos significativos. *ref: He regards women as disposable pleasures rather than as meaningful pursuits.* `out: I consider women to be desirable more than mean goals.` other
12	4.7	Los baños de los aviones no tienen ventanillas por motivos de seguridad. *ref: For security reasons, the bathrooms on airplanes don't have windows.* `out: Airplanes have no windows for safety reasons.` other
13	4.8	Cada vez sube más el precio de la electricidad. *ref: The electric bill keeps going up.* `out: Every time it rises the price of electricity.` no_content_overlap
14	4.8	No hay falsedad en lo que él dice. *ref: What he's saying is true.* `out: There are no falsehood in what he says.` other
15	4.8	Como suele pasar, él no me trajo nada. *ref: As usual he brought me nothing.* `out: As is often the case, he didn't bring me anything.`

#	sBLEU	Source / reference / model output / failure modes
		over_generation
16	4.9	Ella puso en riesgo su vida para salvar a un niño de ahogarse. *ref: She saved the drowning child at the risk of her own life.* `out: She risked her life to save a child from drowning.` other
17	4.9	Tenga cuidado de no resbalar en las baldosas mojadas. *ref: Mind you don't slip on the wet tiles.* `out: Be careful not to slippery.` no_content_overlap
18	4.9	Por favor, ajuste su asiento como le acomode. *ref: Please adjust the seat to fit you.* `out: Please put your seat on as a hold of him.` other
19	4.9	Lo sentimos mucho por no poder ayudarlos. *ref: We are sorry we can't help you.* `out: We're very sorry for not being able to help them.` other
20	4.9	¿Cuál fue el momento en el que te diste cuenta de que habías crecido? *ref: When did you realize you've grown up?* `out: What was the time you noticed that you had grown?` other